מבוא לאבטחת סייבר · 689156 · חוברת ממוקדת

ביטקוין, P2PKH ו-SQL Injection עד הסוף

שלושת הנושאים שהכי קשה לתפוס, מוסברים מאפס ועד הפרטים הקטנים: איך בדיוק עובד הבלוקצ'יין, מי קודם — החתימה או המפתח הציבורי (עם ביצוע הסקריפט על המחסנית), ואיך מרכיבים שאילתת SQL Injection צעד-אחר-צעד. הכל עם דוגמאות פתורות מהמבחנים ותרגול "נסה בעצמך".

חלק 1 · ביטקוין ובלוקצ'יין

הספר (UTXO), בלוקים, עץ מרקל, הוכחת עבודה, forks, 51%, soft/hard fork ומיקרו-תשלומים. כל המלכודות שחוזרות במבחן.

חלק 2 · P2PKH

הסקריפט `DUP HASH160 … CHECKSIG`, ביצוע על המחסנית, ולמה הסדר הוא **חתימה ואז מפתח ציבורי**.

חלק 3 · SQL Injection

איך משרשרים את הקלט לתבנית, למה `' or '1'='1` עובד ו-`and` לא, ותרגול הרכבת שאילתות.

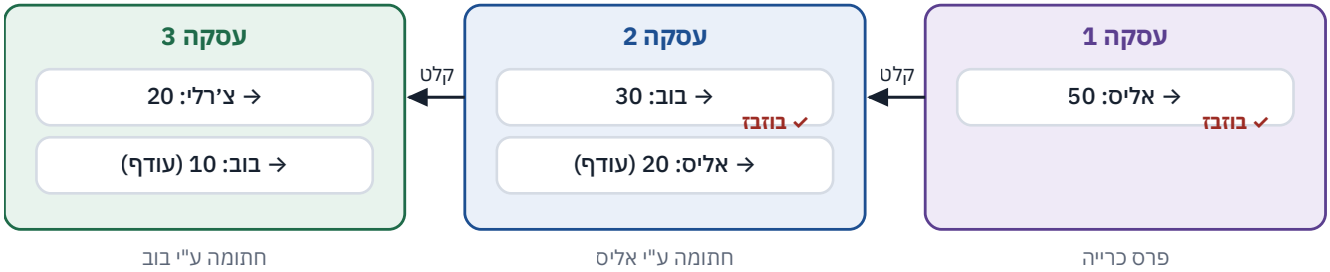
איך ללמוד מהחוברת: קוראים חלק, ואז פותרים את שאלות הנושא באפליקציית התרגול. כל תיבת דוגמה פתורה במלואה, וכל תיבת נסה בעצמך נותנת לכם לפתור לבד — התשובה המלאה ממש מתחת. המטרה: לא להשאיר שום "לא הבנתי".

1 ביטקוין ובלוקצ'יין — מאפס

ביטקוין נראה מבלבל כי יש בו הרבה חלקים, אבל כולם פותרים **בעיה אחת**: איך מעבירים כסף דיגיטלי בלי בנק מרכזי, כך שאי אפשר לזייף ואי אפשר לבזבז את אותו מטבע פעמיים (double spending). נבנה את זה חלק-חלק, וכל חלק יענה על שאלה אחת.

1. איך "יודעים" כמה כסף יש למישהו? מודל ה-UTXO

אין בביטקוין "חשבון" עם יתרה שמורה איפשהו. יש רק **רשימת עסקאות ציבורית** (נקראת ledger — הספר). כל עסקה **מבזבזת פלטים** של עסקאות קודמות, ו**יוצרת פלטים חדשים**. פלט שכבר בוזבז מסומן ולא ניתן לבזבז שוב.



כמה יש לאליס? סוכמים את הפלטים שלה שעוד לא בוזבזו: 20

הספר (ledger) כרשימת עסקאות. אין "יתרות" שמורות בשום מקום: כל עסקה מבזבזת פלטים של עסקאות קודמות ויוצרת פלטים חדשים. פלט שכבר בוזבז מסומן, וניסיון לבזבז אותו שוב הוא double spending.

היתרה של כתובת היא פשוט **סכום הפלטים שלה שעוד לא בוזבזו**. לכן, מי שמחזיק עותק מלא של הבלוקצ'יין (למשל miner) יכול לעבור על כל ההיסטוריה ולחשב כמה ביטקוין יש לכל כתובת ברגע נתון.

מלכודת שחוזרת (Q027)
"miner שמחזיק את כל הבלוקצ'יין יכול לדעת כמה ביטקוין יש לכתובת מסוימת" — **נכון**. הכל ציבורי. מה שלא ידוע הוא מי הבעלים האמיתי של הכתובת (אנונימיות), אבל הסכומים והעסקאות גלויים לכולם.

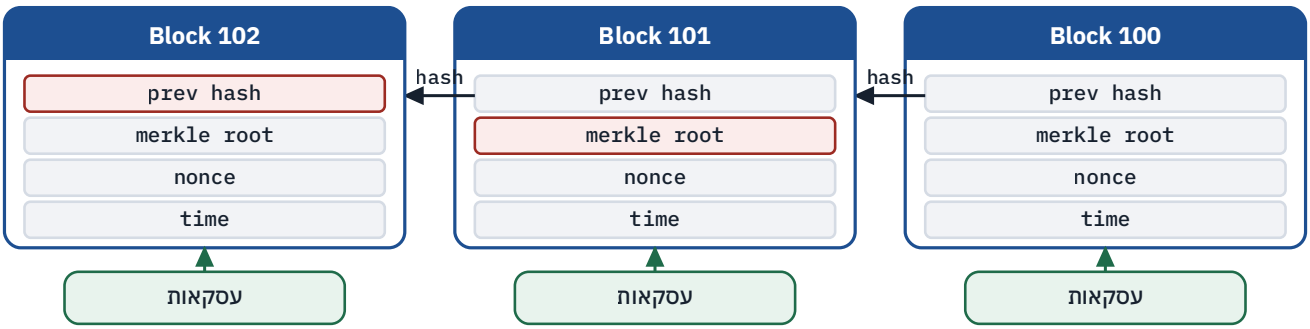
2. מי מחזיק את הכסף? ארנק = זוג מפתחות

- **ארנק** = זוג מפתחות **א-סימטריים** (פרטי + ציבורי). ה**כתובת** נגזרת מ-hash של המפתח הציבורי.
- כדי **לקבל** כסף נותנים את הכתובת. כדי **לבזבז** חותמים על העסקה עם ה**מפתח הפרטי**. רק בעל המפתח הפרטי יכול לבזבז. (איך בדיוק זה נאכף — זה חלק 2, P2PKH.)

מלכודת (Q147, Q018)
בביטקוין **לא** משתמשים ב-One Time Pad כדי לייצר ארנקים. ארנקים הם קריפטוגרפיה א-סימטרית (חתימות), לא OTP סימטרי.

3. איך מחברים עסקאות לשרשרת שלא ניתן לזייף? בלוקים ו-prev hash

העסקאות נארזות ל**בלוקים**. כל בלוק מכיל: `prev hash` (ה-hash של הבלוק הקודם), **שורש עץ מרקל** של העסקאות שבו, nonce וזמן. מכיוון שכל בלוק מצביע ל-hash של הקודם, הם יוצרים **שרשרת**.



שינוי עסקה בבלוק 101 → שורש מרקל חדש → hash חדש → prev hash של 102 לא מתאים

שרשרת בלוקים. כל כותרת מכילה את ה-hash של הכותרת הקודמת ואת שורש עץ המרקל של העסקאות שלה. תוקף ששינה עסקה בבלוק 101 משנה את שורש המרקל, ולכן את ה-hash של בלוק 101, ולכן את ה-prev hash שבבלוק 102, וכן הלאה עד הסוף.

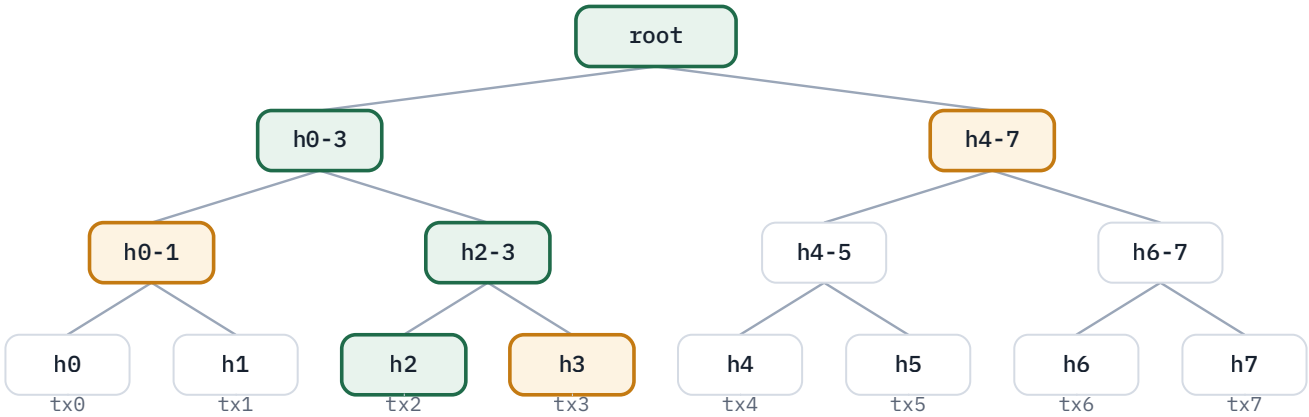
למה זה מונע זיוף? אם תוקף משנה עסקה ישנה, שורש המרקל של אותו בלוק משתנה, ולכן ה-hash של הבלוק משתנה, ולכן ה-`prev hash` של הבלוק הבא כבר לא מתאים, וכן הלאה — **כל השרשרת מאותו בלוק ואילך נשברת**. כדי "לתקן" את זה התוקף היה צריך לחשב מחדש את כל הבלוקים שאחריו (וזה כל הקושי של הוכחת העבודה, סעיף 5).

מלכודת (Q050)

מחשב סטנדרטי שמקבל בלוק חדש ומוסיף אותו לעותק המקומי — **לא** מוסיף חתימה משלו לבלוק. הוא רק **מאמת** את הבלוק (עסקאות חתומות, אין double spending, ה-hash מתחיל במספיק אפסים, ה-prev hash מתאים) ומוסיף אותו כמו שהוא.

4. עץ מרקל: לאמת עסקה בלי להחזיק את כל הבלוק

במקום לשמור בבלוק את כל העסקאות, שומרים רק את **שורש עץ המרקל** שלהן. עץ מרקל הוא עץ בינארי: העלים הם ה-hash-ים של העסקאות, וכל צומת הוא `h(ילד שמאלי || ילד ימני)`, עד לשורש אחד.



עץ מרקל והוכחת הכללה (הדוגמה מההרצאה). כדי להוכיח ש-h_2 בעץ מספיקים שלושה ערכים (בכתום): h_3, h_{0-1} ו-h_{4-7}. המאמת מחשב בעצמו את הצמתים הירוקים ומשווה לשורש. עם n עלים צריך $\log_2 n$ ערכים.

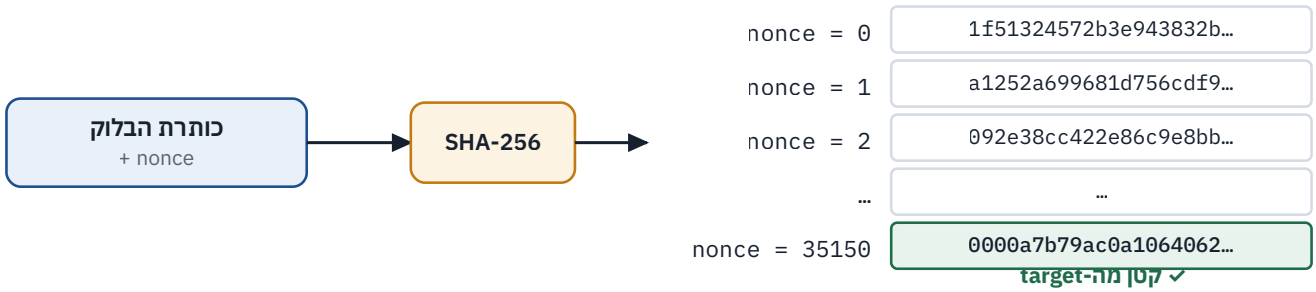
הוכחת הכללה (proof of inclusion): לקוח "קל" שלא מחזיק את כל העץ רוצה לוודא שעסקה מסוימת בבלוק. הוא מבקש מהצומת המלא רק את ה-hash-ים של ה"אחים" לאורך המסלול מהעלה עד השורש — כלומר $\log_2(n)$ ערכים — מחשב בעצמו את הדרך עד השורש, ומשווה לשורש שבבלוק.

שלוש שאלות מרקל שחוזרות — תדעו אותן בעל פה

- **Q159:** עץ עם **1024** עלים, כמה hash-ים צריך לקוח קל להוכחת הכללה? $\log_2 1024 = 10$.
- **Q152:** משנים **עלה אחד** — אילו צמתים (לא-עלים) משתנים? **רק הצמתים בנתיב מהעלה עד השורש, כולל השורש**. לא כל העץ, ולא רק האבא הישיר.
- **Q079:** "עץ מרקל מבטיח **פרטיות** כך שרק המשתתפים רואים את העסקה" — **שגוי**. מרקל נותן אימות יעיל ושלמות, **לא פרטיות ולא הצפנה**. כל העסקאות ציבוריות.

5. איך מוסיפים בלוק? הוכחת עבודה (Proof of Work)

כדי שלא כל אחד יוסיף בלוקים כרצונו, מחייבים את הכורה לפתור "חידה" יקרה: למצוא nonce (מספר) שכאשר מגבבים את כותרת הבלוק יחד איתו, ה-hash שמתקבל **מתחיל במספיק אפסים** (קטן מ-target). הדרך היחידה למצוא nonce כזה היא **ניסוי וטעייה**, אבל **לאמת** פתרון זה חישוב של hash **אחד**.



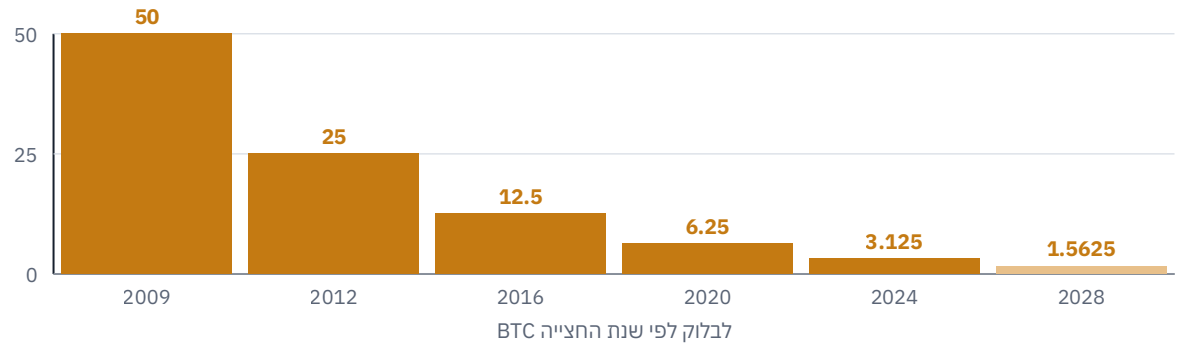
הוכחת עבודה בפועל (חישוב אמיתי). הכורה משנה את ה-nonce שוב ושוב ומחשב SHA-256 על כותרת הבלוק. במקרה הזה נדרשו 35,150 ניסיונות כדי לקבל hash שמתחיל בארבעה אפסים הקסדצימליים. כל מי שרוצה לבדוק מחשב hash אחד.

איפה משתמשים ב-nonce? (Q029)

ה-nonce הוא המספר שהכורה משנה שוב ושוב **בתהליך החיפוש אחר בלוק חדש**. זה כל תפקידו.

מספרים שחייבים לזכור:

```
block every ~10 min · difficulty retargets every 2016 blocks · reward halves every
210,000 blocks · max supply 21,000,000 BTC
```



פרס הבלוק נחצה כל 210,000 בלוקים (כ-4 שנים). סכום הסדרה `210,000 × 100 = 210,000 × (50 + 25 + 12.5 + …)` שווה ל-`21,000,000`, ולכן יהיו לכל היותר 21 מיליון ביטקוין. בסביבת 2140 הפרס יגיע לאפס, והכורים יחיו מעמלות.

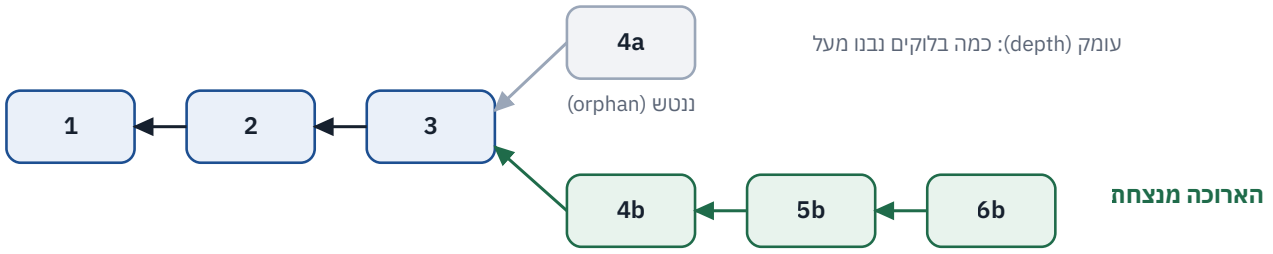
- **הפרס:** הכורה שמצא בלוק מקבל פרס (בעסקה הראשונה, coinbase) ועוד את העמלות. הפרס נחצה כל 210,000 בלוקים (כ-4 שנים): `50 → 25 → 12.5 → 6.25 → …`, עד שבסביבות 2140 יגיע לאפס.
- **mining pools (Q042):** כורה בודד כמעט לא ימצא בלוק, ולכן כיום כורים בעיקר ב**קבוצות כרייה** שמחלקות את העבודה ואת הפרס.

המלכודת הכי חשובה בביטקוין (Q046)

הצעה: "כדי לחסוך חשמל, נכרה בלוקים רק כשיש לפחות x עסקאות חדשות". זו הטענה **השגויה**. בזמן שהכורים הישרים "מחכים" לעסקאות, תוקף יכול להמשיך לכרות בשקט שרשרת סודית, לצבור יתרון, ואז לפרסם אותה — וזה פותח פתח ל-double spending. קצב קבוע (PoW תמידי) הוא מה שמגן.

6. מי צודק כשיש מחלוקת? קונצנזוס, forks ו-double spending

לפעמים שני כורים מוצאים בלוק כמעט באותו זמן, ונוצר **fork** (פיצול). הכלל: **השרשרת הארוכה ביותר מנצחת** (ליתר דיוק, זו עם הכי הרבה עבודה). כל צומת ממשיך על הענף הארוך, והענף הקצר נטש (orphan).



Fork וכלל השרשרת הארוכה. שני כורים מצאו בלוק 4 כמעט באותו זמן. כל צומת ממשיך על הענף שהגיע אליו ראשון, עד שענף אחד מתארך. הענף הקצר נגטש, והעסקאות שהיו רק בו חוזרות להמתנה. לכן מחכים כ-6 בלוקים לפני שסומכים על עסקה.

איך זה מונע double spending? (Q137) כדי לבזבז מטבע פעמיים, תוקף היה צריך לבנות שרשרת **ארוכה יותר** מכל שאר הרשת ביחד — ואין לו מספיק כוח חישוב. לכן מחכים **כמה בלוקים** (בד"כ 6) מעל העסקה: אם היא נשארה בשרשרת הארוכה אחרי 6 בלוקים, היא "סגורה".

Q053 — אליס מנסה לשכתב היסטוריה

אליס משנה את העותק המקומי שלה כדי "לבטל" עסקה. הטענה הנכונה: **כל עוד השרשרת שאליס מציעה אינה ארוכה יותר מהמקורית, סיכוייה להצליח נמוכים**. אין "בנקאי" שמבטל — רק אורך השרשרת קובע.

51% (מי ששולט ברוב כוח החישוב)

יכול: לקבוע אילו עסקאות ייכנסו, למנוע מעסקאות מסוימות להיכנס (exclusion), ולבצע double spending. **לא** יכול: לגנוב מטבעות של אחרים (אין לו את המפתחות הפרטיים שלהם). **Sybil** (המון זהויות מזויפות) לא עוזר, כי ב-PoW מה שקובע זה כוח חישוב ולא מספר זהויות.

Q127 — לקוח "רזה"

לקוח ששומר רק את העסקאות שלא בוזבזו ואת הבלוק האחרון **לא יכול להתמודד בעצמו עם חלק מה-forks**, כי אין לו את ההיסטוריה המלאה כדי לחזור אחורה ולבחור ענף.

7. שינויים בפרוטוקול: Soft fork מול Hard fork

Soft fork — הכללים מוחמרים	Hard fork — הכללים משתנים/מורחבים
בלוקים חדשים עדיין חוקיים בעיני צמתים ישנים. דוגמה: הקטנת גודל בלוק מ-2MB ל-1MB (בלוק של 1MB חוקי גם לפי הכללים הישנים)	צמתים ישנים **דוחים** את הבלוקים החדשים, והרשת מתפצלת לשני מטבעות. דוגמאות: הגדלת בלוק (Bitcoin Cash), או **החלפת פונקציית ה-hash של ה-PoW**

Q072

"משנים את פונקציית ה-hash של ה-PoW" → זו דוגמה ל-**hard fork** (צמתים ישנים לא יכירו בבלוקים החדשים).

8. עסקאות מתקדמות: escrow ומיקרו-תשלומים



Escrow עם multisig של 2 מתוך 3. הכסף יושב בארנק משותף. כל שתי חתימות משחררות אותו. בעסקה תקינה הקונה והמוכר חותמים לבד; המתווך נכנס רק בסכסוך.

- **multisig / Escrow (2 מתוך 3):** קונה, מוכר וצד-שלישי מוסכם חולקים ארנק שדורש 2 חתימות. בעסקה תקינה — **הקונה והמוכר** חותמים לבד, בלי המתווך. בסכסוך — המתווך מכריע עם אחד הצדדים. לכן "המתווך **חייב** לאשר" — **שגוי** (Q018).
- **מיקרו-תשלומים + LockTime (Q018):** אליס נועלת כסף בארנק משותף עם בוב ומחזיקה מראש עסקת-החזר עם LockTime (עסקה שלא תיכנס לבלוק לפני זמן מסוים). כל תשלום קטן הוא עסקה חדשה שאליס חותמת, ובוב מפרסם רק את האחרונה. **אליס יודעת שבוב ישחרר לה את מה שמגיע לה — כי אחרת, כשה-LockTime יגיע, כל הכסף יחזור אליה ובוב יפסיד.**

נסה בעצמך

עץ מרקל בבלוק מכיל **256** עסקאות. (א) כמה hash-ים צריך לקוח קל כדי להוכיח שעסקה אחת כלולה? (ב) אם משנים עסקה אחת, כמה צמתים לא-עלים משתנים?

תשובה: (א) $\log_2 256 = 8$ hash-ים. (ב) רק הצמתים ב**נתיב מהעלה עד השורש**: בעץ של 256 עלים הגובה הוא 8, כלומר **8 צמתים לא-עלים** (כולל השורש).

סיכום חלק 1 — תגיד לעצמך בקול

- אין יתרות, יש פלטים שבוזבזו/לא בוזבזו (UTXO). ארנק = מפתחות; כתובת = hash של הפומבי; אין OTP.
- בלוק = prev hash + שורש מרקל + nonce. שינוי ישן שובר את כל השרשרת אחריו. מחשב מאמת ולא חותם.
- מרקל: הוכחה ב-$\log_2 n$; שינוי עלה → נתיב לשורש; לא פרטיות.
- PoW: nonce בניסוי-וטעייה; 10 דק'; קושי כל 2016; פרס נחצה כל 210,000; 21M.
- השרשרת הארוכה מנצחת; מחכים 6 בלוקים; 51% שולט בקבלה אך לא גונב; "לכרות רק עם x עסקאות" = טעות.
- soft fork = החמרה (תואם אחורה); hard fork = שינוי hash / הגדלת בלוק.

2 P2PKH — מי קודם, החתימה או המפתח?

זו השאלה שמבלבלת, ויש לה תשובה חד-משמעית: ב-input script הסדר הוא **קודם החתימה, אחר כך המפתח הציבורי** — `[Signature] [Public Key]`. הסעיף הזה יסביר **למה** זה הסדר, דרך ביצוע הסקריפט על המחסנית. אחרי שתבין את הריצה פעם אחת, לא תטעה בזה שוב.

מה הבעיה שהסקריפט פותר?

כשמישהו שולח לך ביטקוין, הפלט "ננעל" לכתובת שלך. הכתובת היא **hash של המפתח הציבורי** שלך (לכן השם P2PKH = Pay to Public-Key-Hash). כדי לבזבז את הכסף הזה בהמשך, עליך להוכיח לכורים שני דברים:

1. שאתה באמת הבעלים של הכתובת — כלומר שיש לך מפתח ציבורי ש-hash שלו שווה לכתובת.
2. שאתה באמת שולט במפתח — כלומר שחתמת על העסקה עם ה**מפתח הפרטי** המתאים.

שים לב שאלה בדיוק שתי הדרישות: הראשונה קושרת אותך לכתובת (hash), השנייה מוכיחה שליטה (חתימה). לכן בפדיון חייבים לספק **גם** את המפתח הציבורי **וגם** את החתימה.

שני חצאי הסקריפט

בביטקוין, "הנעילה" וה"פתיחה" הן שתי תוכניות קטנות בשפת סקריפט פשוטה שרצה על **מחסנית** (stack). יש שני חצאים:

חצי	מי כותב אותו	התוכן ב-P2PKH
output script (הנעילה)	מי ששלח לך את הכסף, בעבר	`DUP HASH160 [Key Hash] EQUALVERIFY CHECKSIG`
input script (הפתיחה)	**אתה**, כשאתה מבזבז	`[Signature] [Public Key]`

כדי לבדוק אם הבזבוז חוקי, מריצים את **שני החצאים ברצף**: קודם ה-input script ואז ה-output script. כלומר הסקריפט המלא שרץ הוא:

```
[Signature] [Public Key] DUP HASH160 [Key Hash] EQUALVERIFY CHECKSIG
```

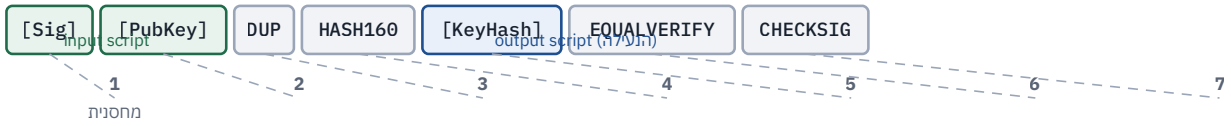
(בכתום-כהה שלך, באפור של הנעילה.) עכשיו נריץ אותו על המחסנית פקודה-אחר-פקודה.

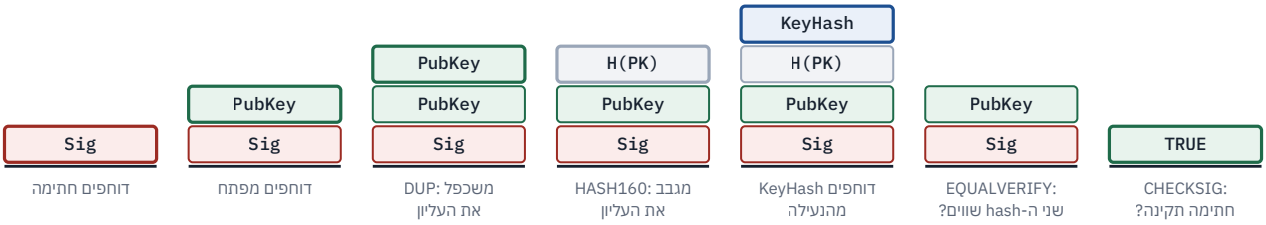
מה עושה כל פקודה?

פקודה	מה היא עושה על המחסנית
`[X]` (ערך)	**דוחפת** את הערך X לראש המחסנית
`DUP`	**משכפלת** את הערך שבראש המחסנית (עכשיו הוא מופיע פעמיים)
`HASH160`	לוקחת את הערך שבראש, **מגבבת** אותו, ומחליפה אותו בתוצאת ה-hash
`EQUALVERIFY`	**שולפת שניים** מהראש ובודקת שהם שווים. אם לא — הסקריפט **נכשל מיד**. אם כן — ממשיכים (השניים הוסרו)
`CHECKSIG`	**שולפת שניים**: מפתח ציבורי וחתימה, ובודקת שהחתימה תקינה על העסקה מול המפתח. דוחפת TRUE אם תקין

הריצה המלאה — כאן נופל האסימון

הסקריפט המלא (input ואחריו output):





ביצוע סקריפט ה-P2PKH על המחסנית (stack), צעד-אחר-צעד. קוראים את הסקריפט משמאל לימין ודוחפים/מעבדים על מחסנית. שימו לב **למה** החתימה נדחפת ראשונה (היא בתחתית, ומשמשת אחרונה ב-CHECKSIG) והמפתח הציבורי שני (הוא למעלה, ומשמש ראשון ב-DUP/HASH160). זה כל הסוד של "מי קודם".

1. `[Signature]` — דוחפים את החתימה. היא נמצאת עכשיו ב**תחתית** המחסנית.
2. `[Public Key]` — דוחפים את המפתח הציבורי. הוא עכשיו ב**ראש**, מעל החתימה.
3. `DUP` — משכפלים את הראש (המפתח הציבורי). יש שני עותקים של המפתח.
4. `HASH160` — מגבבים את העותק העליון של המפתח. מתקבל `h(PubKey)`.
5. `[Key Hash]` — דוחפים את ה-hash ששמור בנעילה (הכתובת שאליה הכסף ננעל).
6. `EQUALVERIFY` — משווים: `h(PubKey) == Key Hash?` אם לא — נכשל (המפתח לא תואם לכתובת). אם כן — ממשיכים. **זו דרישה 1: הוכחת שייכות לכתובת.**
7. `CHECKSIG` — נשארו בדיוק **מפתח ציבורי וחתימה.** בודקים שהחתימה תקינה מולם. **זו דרישה 2: הוכחת שליטה.**

עכשיו ברור "מי קודם"

למה חתימה ראשונה ומפתח ציבורי שני

המחסנית היא "אחרון-נכנס-ראשון-יוצא". המפתח הציבורי צריך להיות **זמין ראשון** (עבור DUP ו-HASH160 בתחילת הנעילה), ולכן הוא חייב להיות ב**ראש** — כלומר להידחף **אחרון**. החתימה נדרשת רק ב**סוף** (ב-CHECKSIG), ולכן היא יכולה להיות ב**תחתית** — כלומר להידחף **ראשונה**. מכאן הסדר: `[Signature] [Public Key]`.

הדוגמאות מהמבחן — פתורות

דוגמה · Q058

נתונה הנעילה `DUP HASH160 [Key Hash] EQUALVERIFY CHECKSIG`. מהו input script שמאפשר לבזבז?

✓ **התשובה:** `[Signature] [Public Key]`. למה לא האחרים?

- `CHECKSIG EQUALVERIFY` — אלה **פקודות** של הנעילה, לא ערכים שהמשתמש מספק.
- `[Public Key] [Key Hash]` — אסור לספק את ה-Key Hash (הוא כבר בנעילה), וגם אין חתימה.
- `[Key Hash] [Public Key]` — אין חתימה, אז CHECKSIG ייכשל.

דוגמה · Q087 — הסקריפט המחובר התקין

צריך לבחור את הסקריפט המלא (input+output) ה**חוקי**:

✓ `[Signature] [Public Key] DUP HASH160 [Key Hash] EQUALVERIFY CHECKSIG`

- `… EQUALVERIFY` בלי `CHECKSIG` — **פסול**: בדקנו רק שהמפתח תואם לכתובת, אבל **לא** אימתנו את החתימה. כל אחד שיודע את המפתח הציבורי (הוא גלוי!) היה יכול לבזבז.
- `… [Public Key] [Signature] HASH160 DUP` — **פסול**: הסדר הפוך (מפתח לפני חתימה) וגם HASH160 לפני DUP.
- סקריפט שמתחיל ב-`CHECKSIG` — **פסול**: אין עדיין כלום על המחסנית לבדוק.

דוגמה · Q162 — מה בדיוק המשתמש מספק

✓ **התשובה:** את ה-**public key** של הארנק, ו**חתימה שנחתמה עם ה-private key**.

שתי הטעויות הקבועות במסיחים: (1) לספק את ה-**hash** של המפתח במקום את המפתח עצמו — **לא**, ה-hash כבר רשום בנעילה, וצריך את המפתח המלא כדי ש-HASH160 יחשב אותו. (2) "חתימה שנחתמה עם ה-**public key**" — **אי אפשר לחתום עם מפתח ציבורי**; חותמים רק עם הפרטי.

נסה בעצמך

חבר מציע input script כזה: `[Public Key] [Signature]`. האם העסקה תיחשב חוקית? תסביר לפי המחסנית.

תשובה: לא. בסדר הזה, אחרי הדחיפות המחסנית היא (מלמטה) `PubKey, Signature`, כלומר ה**חתימה** בראש. הפקודה הבאה `DUP` תשכפל את החתימה (לא את המפתח), `HASH160` תגבב את החתימה, וההשוואה ל-Key Hash תיכשל. הסדר **חייב** להיות חתימה-ואז-מפתח, כי המפתח צריך להיות בראש עבור DUP/HASH160.

סיכום חלק 2 — המשפט שתזכור

- בפדיון מספקים **מפתח ציבורי + חתימה (שנחתמה בפרטי)**. לא את ה-hash, ולא "חתימה בפומבי".
- סדר ה-input: `[Signature] [Public Key]` — חתימה ראשונה (בתחתית, משמשת אחרונה), מפתח שני (בראש, משמש ראשון).
- הנעילה: `DUP HASH160 [Key Hash] EQUALVERIFY CHECKSIG`. EQUALVERIFY = תואם לכתובת; CHECKSIG = חתימה תקינה. בלי CHECKSIG — פסול.

3 SQL Injection — להרכיב את השאילתה

כל הקושי כאן הוא מכני: לקחת את הקלט של המשתמש, "להדביק" אותו לתוך התבנית של השרת, ולקרוא מה יצא. ברגע שתתרגל את ההדבקה הזאת כמה פעמים — תזהה מיד אם קלט מסוים הוא injection תקין. נבנה את המיומנות הזאת צעד-אחר-צעד.

מאיפה בכלל מגיעה הבעיה?

השרת בונה את השאילתה ב**שרשור מחרוזות**: יש חלק קבוע שהשרת כתב, ובאמצע "נתקע" מה שהמשתמש הקליד. זו התבנית המדויקת מהמבחן (Q060, Q095):

```
"SELECT * FROM users WHERE username = '" + InputUsername + "' AND password = '" +
InputPassword + "'"
```

שימו לב לגרשים הבודדים ' שהשרת שם **מסביב** לקלט — הם אמורים לסמן "כאן מתחילה מחרוזת, כאן היא נגמרת". הרעיון של ההתקפה: הקלט עצמו מכיל **גרש**, שסוגר את המחרוזת **מוקדם**, וכל מה שבא אחריו כבר לא נקרא כנתון אלא **כחלק מהלוגיקה של השאילתה**.

איך "מרכיבים" — שיטה של שלושה צעדים

1. **העתק את התבנית** בדיוק כמו שהיא, עם הגרשים הקבועים של השרת.
2. **הדבק את הקלט** במקום `InputUsername` ו-`InputPassword` — תו-בתו, בלי לשנות כלום.
3. **קרא את התוצאה** כאילו אתה מסד הנתונים: האם נוצר תנאי שתמיד אמת? האם הסיסמה עדיין נבדקת?



איך מרכיבים את השאילתה (הדוגמה המדויקת מהמבחן). השרת משרשר את מה שהמשתמש הקליד לתוך תבנית קבועה. עוקבים: מורידים את הקלט למקומו, ורואים מה יוצא. בירוק: החלקים הקבועים של השרת. בכתום: מה שהמשתמש שתל.

הדוגמה המרכזית, שלב-שלב (Q060)

הקלט הזוכה: `InputUsername = alice`, `InputPassword = 1234' or '1' = '1`. נדביק:

```
SELECT * FROM users WHERE username = 'alice' AND password = '1234' or '1' = '1'
```

עכשיו נקרא את תנאי ה-`WHERE`. בגלל סדר הקדימויות (`AND` לפני `OR`), התנאי מתפרק כך:

```
( username='alice' AND password='1234' ) OR '1' = '1'
```

הצד הראשון אולי שקרי (הסיסמה 1234 כנראה לא נכונה), אבל הצד השני, `'1' = '1'`, **תמיד אמת**. ו-OR מחזיר אמת אם **לפחות צד אחד** אמת. לכן כל שורה בטבלה "עוברת" את התנאי, השאילתה מחזירה שורות, והשרת מחבר אותנו — **בלי לדעת את הסיסמה**.

הדרך השנייה: הערה שמוחקת את הסוף (Q095)

הקלט: `InputUsername = ' or 1=1 --`. בשפת SQL, `--` הוא **תחילת הערה**: כל מה שאחריו באותה שורה **מתעלמים ממנו**. נדביק:

```
SELECT * FROM users WHERE username = '' or 1=1 --' AND password = '…'
```

הגרש הראשון של הקלט סוגר את המחרוזת הריקה, `or 1=1` הופך את התנאי לאמת-תמיד, ו-`--` **מבטל את כל בדיקת הסיסמה** שבאה אחר כך. שוב מתחברים בלי סיסמה. (בשאלה הזו גם ה-password מכיל injection תקין, אבל ה-username לבדו כבר מספיק.)

למה חלק מהקלטים נכשלים — לזהות את המסיחים

הקלט (ב-password)		למה
`1234' or '1' = '1`	✓	גרש סוגר, OR עם '1'='1' (אמת) → עובר
`' or 1=1 --`	✓	גרש סוגר, OR 1=1, ו---- מבטל את השאר → עובר
`1234' and '1' = '1`	✗	דורש גם סיסמה נכונה → נכשל AND
`1234' and "1" = "1`	✗	ב-SQL מחרוזת היא בגרש בודד, לא גרשיים → שגיאת תחביר
`or 1=1`	✗	בלי גרש פותח: נשאר בתוך המחרוזת, לא קוד

למה חלק מהקלטים עובדים וחלק לא. ההבדל הקריטי: (1) האם הגרש "בורח" מהמחרוזת? (2) האם משתמשים ב-OR (שמספיק שצד אחד אמת) ולא ב-AND (שדורש את שני הצדדים)? (3) ב-SQL מחרוזת היא בגרש בודד `'`, ולא בגרשיים כפולים `"`.

שלושת הדברים שמפילים מסיחים, כל אחד הוא שאלה שתשאל את עצמך:

1. **האם הגרש "בורח" מהמחרוזת?** בלי גרש שסוגר את המחרוזת של השרת, הקלט נשאר **בתוך** המחרוזת ונחשב לטקסט רגיל, לא לקוד. לכן `or 1=1` בלי גרש פותח — **נכשל**.
2. **OR או AND?** כדי **לעקוף** את בדיקת הסיסמה צריך OR עם משהו שתמיד אמת — מספיק שצד אחד אמת. עם AND התנאי דורש **גם** את הסיסמה הנכונה, אז זה **לא עוקף** כלום. לכן `1234' and '1'='1` — **נכשל** כעקיפה.
3. **גרש בודד או כפול?** ב-SQL מחרוזת עטופה ב**גרש בודד** `'`. גרשיים כפולים `"` הם לא תיחום מחרוזת ב-SQL (לרוב שם של עמודה) ויגרמו ל**שגיאת תחביר**. לכן `and "1"="1` — **נכשל**.

נסה בעצמך 1

אותה תבנית. הקלט: `InputUsername = admin' --`, שום דבר (לא משנה) `InputPassword =`. האם מתחברים? כמי?

תשובה: כן, כ-admin. נדביק: `WHERE username = 'admin' --' AND password = '…'` …. הגרש סוגר את השם `admin`, ו-`--` מוחק את כל בדיקת הסיסמה. השאילתה מחזירה את שורת ה-admin, ומתחברים כמנהל **בלי סיסמה**. (זו הסיבה שבטופס אמיתי זו פרצה חמורה במיוחד — בוחרים משתמש ספציפי.)

נסה בעצמך 2

האם הקלט `InputPassword = 1234" or "1"="1` (עם גרשיים **כפולים**) יעבוד?

תשובה: לא. הגרש הכפול `"` לא סוגר את המחרוזת שהשרת פתח בגרש בודד `'`. הקלט נשאר תקוע בתוך המחרוזת ו/או גורם לשגיאת תחביר. צריך גרש **בודד** כדי לברוח.

עוד שתי יכולות שחוזרות

- **Blind SQL Injection (Q035):** התוקף **לא רואה** את הפלט של השאילתה, אבל שואל שאלות כן/לא ומסיק את התשובה בעקיפין — למשל לפי האם הדף מוצג או לא, או לפי **כמה זמן** לוקח לשרת לענות. כך מגלים סיסמה תו-אחר-תו.
- **עוד מה שאפשר להזריק:** לסיים שאילתה ולהתחיל אחרת עם `;` (למשל `'; DROP TABLE users --`), לשלוף מטבלה אחרת עם `UNION`, ולעקוף סינון מבוסס **רשימה שחורה** (למשל `uniUNIONon` — אחרי מחיקת מופע אחד של המילה האסורה נשארת המילה האסורה).

ההגנות (וגם זה נשאל)

הפתרון השורשי: לא לשרשר קלט לתוך קוד השאילתה.

- **שאילתות עם פרמטרים** (prepared statements / stored procedures): מבנה השאילתה מוגדר מראש, והקלט נכנס כ**ערך** נפרד — לעולם לא כקוד. גם אם הקלט מכיל גרשיים, הם נשארים חלק מהמחרוזת.
- בנוסף: ORM, **ולידציית קלט בצד השרת** (חובה; בצד הלקוח רק נוחות), חיטוי וקידוד (escaping/encoding), ואי-הצגת הודעות שגיאה למשתמש.

מלכודות שחוזרות

- **Q017:** מה **לא** מונע injection? **IDS**. (חיטוי קלט, סינון ו-WAF כן מונעים/מצמצמים.)
- **Q109:** שם שמוצג כ-`'` במקום גרש = האתר משתמש ב**קידוד (encoding)** כדי למנוע injection.
- **Q102:** `X-Frame-Options` נועד נגד **Clickjacking**, **לא** נגד SQL injection.
- **הודעות כניסה כלליות:** תמיד "שם משתמש **או** סיסמה שגויים", לא לחשוף איזה מהם.

סיכום חלק 3 — תהליך קבוע

- הדבק את הקלט לתבנית תו-בתו → קרא את תנאי ה-WHERE.
- תקין אם: (1) גרש **בודד** סוגר את המחרוזת, (2) `OR` עם תנאי אמת-תמיד או `--` שמוחק את ההמשך.
- נכשל אם: אין גרש פותח / משתמשים ב-`AND` / גרשיים כפולים.
- הגנה שורשית: **שאילתות עם פרמטרים** (הקלט תמיד ערך, לא קוד). IDS לא מונע.

4 דף מרוכז — יום לפני המבחן

כל מה שצריך לזכור משלושת החלקים, בעמוד אחד. אם משהו כאן לא ברור — חזור לפרק המתאים.

ביטקוין — מספרים ומלכודות

נושא	מה לזכור
מספרים	בלוק כל ~10 דק' · קושי מתעדכן כל **2016** בלוקים · פרס נחצה כל **210,000** · תקרה **21M**
מרקל	הוכחת הכללה = $\log_2 n$ hash-ים (1024→10, 256→8). שינוי עלה → רק הנתיב לשורש. **לא** פרטיות
nonce	המספר שמחפשים בניסוי-וטעייה כדי למצוא בלוק (PoW)
קונצנזוס	השרשרת **הארוכה** מנצחת · מחכים ~6 בלוקים · שינוי ישן שובר את כל השרשרת אחריו
51%	**כן:** קובע מי נכנס, double spending, exclusion. **לא:** גונב מטבעות (אין מפתחות). Sybil לא עוזר
forks	soft = החמרה/תואם אחורה (בלוק 2MB→1MB) · hard = שינוי hash / הגדלת בלוק
מחשב סטנדרטי	**מאמת** בלוק ומוסיף אותו — **לא** מוסיף חתימה משלו
escrow / micro	2-of-3: קונה+מוכר חותמים לבד; מתווך רק בסכסוך. LockTime: אחרת הכסף חוזר לאליס
הטעות הקלאסית	"לכרות רק כשיש x עסקאות כדי לחסוך חשמל" — **שגוי** (פותח double spending)
ארנק	מפתחות א-סימטריים; כתובת = hash של הפומבי; **אין OTP**

P2PKH — הסקריפט על המחסנית

```
input: [Signature] [Public Key] output: DUP HASH160 [Key Hash] EQUALVERIFY CHECKSIG
```

שלב	מחסנית (מלמטה למעלה)
1. [Signature]	`Sig`
2. [Public Key]	`Sig · PubKey`
3. DUP	`Sig · PubKey · PubKey`
4. HASH160	`Sig · PubKey · h(PubKey)`
5. [Key Hash]	`Sig · PubKey · h(PubKey) · KeyHash`
6. EQUALVERIFY	בודק `h(PubKey)=KeyHash` (שייכות) → נשאר `Sig · PubKey`
7. CHECKSIG	בודק חתימה (שליטה) → `TRUE`

המשפט
חתימה **ראשונה** (בתחתית, משמשת אחרונה ב-CHECKSIG), מפתח ציבורי **שני** (בראש, משמש ראשון ב-DUP/HASH160).
מספקים **מפתח + חתימה בפרטי**, לא hash ולא "חתימה בפומבי". בלי CHECKSIG בסוף — פסול.

SQL Injection — הרכבה בשלושה צעדים

```
template: … username = '__user__' AND password = '__pass__'
```

1. **הדבק** את הקלט תו-בתו במקום __user__ ו-__pass__.
2. **קרא** את ה-WHERE: נוצר תנאי אמת-תמיד? הסיסמה עדיין נבדקת?
3. **הכרעה:**

✓ עובד	× נכשל
גרש **בודד** שסוגר + `or '1'='1`	`and` … (דורש סיסמה נכונה)
גרש + `-- or 1=1` (מוחק את הסוף)	בלי גרש פותח (נשאר טקסט)
	גרשיים **כפולים** " (שגיאת תחביר)

מהיר

IDS **לא** מונע injection · הגנה שורשית = שאילתות עם פרמטרים · ' = קידוד נגד injection · `X-Frame-Options` = נגד clickjacking, לא Blind · injection = מסיקים כן/לא לפי התנהגות/זמן.

שלוש השאלות שתשאל את עצמך בכל שאלה

- **ביטקוין:** "האם זה שובר את כלל השרשרת הארוכה / הקונצנזוס?" רוב המלכודות נופלות כאן.
- **P2PKH:** "מה על המחסנית עכשיו, ומה הפקודה הבאה צריכה שם?" הסדר נגזר מזה.
- **SQL:** "הדבקתי את הקלט — מה בדיוק ה-WHERE אומר עכשיו?"

בהצלחה במבחן 🎓 — עכשיו לך תפתור את שאלות ביטקוין ו-Web באפליקציה, ותראה שהן כבר לא מפחידות.